



Tools and Reference

01/25/2026

1. Falcon Query Language (FQL)

1.1. Introduction

1.2. Filtering with FQL

- 1.2.1. Entering multiple values for a filter
- 1.2.2. Using multiple filters in a single request
- 1.2.3. Properties
- 1.2.4. Operators (optional)
- 1.2.5. Values
 - 1.2.5.1. Special characters

1.3. Sorting with FQL

1.4. Combining FQL expressions

1.5. Using timestamps

- 1.5.1. Searching partial timestamp values
- 1.5.2. Using comparison operators with timestamps

1.6. Use cases and examples

- 1.6.1. Hosts API
- 1.6.2. Kernel support info API

Falcon Query Language (FQL)

Query, sort, and filter your Falcon data with a SQL-like query language.

Introduction

Falcon Query Language (FQL) is used to query, sort, and filter your Falcon data. For example, you might use an FQL expression to sort the list of detections in the Falcon console or to filter the hosts you retrieve in a CrowdStrike API request.

FQL is a SQL-like query language, so if you already understand SQL expressions, you'll find FQL familiar.

Filtering with FQL

An FQL filter expression is a string in this format: **<property>:<operator><value>**

- **<property>** represents the item you want to filter on, such as an attribute of a detection (**status**) or a host (**platform_name**)
- **:** is required and separates the property from the operator and value
- **<operator>** is an optional field used to compare values, such as **!** or **<=**.
- **<value>** is the value for the property, such as a boolean (**true** or **false**), a number (such as **1** or **42**), a string, or **null**

Examples of valid filter expressions:

- `platform_name: 'Windows'`
- `status: 'normal'`
- `last_seen: >='2016-07-04'`

Entering multiple values for a filter

In filters that feature multi-value support, enter multiple values as a comma-separated list enclosed in brackets.

- `cve.severity: ['HIGH', 'CRITICAL']`

Using multiple filters in a single request

To narrow your result set, you can use multiple filters in requests by separating them with a URL-encoded **+** operator (**%2B**).

- `created_timestamp:>'2021-10-02'%2Bfilter=cve.exploit_status:'60'`

Note: The **&** operator combines multiple parameters in a request. When **&** is used between two filters, only the first filter is used.

Properties

Properties are the element of the Falcon data that you want to filter on. Properties always contain only alphanumeric characters and underscores (_). The first character is always a letter, and properties are always considered lowercase (though uppercase characters are accepted and automatically converted). Some names for complex properties are separated by periods (.), such as **author.name** or **posts.count**.

Operators (optional)

By default, an expression's operator is "equal to." For example, **platform_name: 'windows'** returns hosts where the attribute **platform_name** is equal to **windows**. Optionally, you can modify the operator of an FQL expression by adding one of these after the colon (:)

Operator	Character name	Description
!	Exclamation point	not equal to
>	Greater than sign	greater than
>=	Greater than sign and equal sign	greater than or equal to
<	Less than sign	less than
<=	Less than sign and equal sign	less than or equal to
~	Tilde	text match. Tokenizes the string, ignoring spaces, case, and punctuation.
!~	Exclamation point and tilde	does not text match. Tokenizes the string, ignoring spaces, case, and punctuation.
*	Asterisk	wildcard matching. Use both an operator asterisk and one or more wildcard asterisks.
~*	Tilde and asterisk	case-insensitive wildcard text contains
~*!	Tilde and asterisk and exclamation point	case-insensitive wildcard text does not contain

Keep these two things in mind when using wildcard matching:

- Indicate that a search is a wildcard search instead of the default "is equal to" by adding the operator asterisk (*) to the beginning of the search string.
- Place wildcard asterisks (*) where you want to match one or more characters.

Values

An FQL expression's value is the data item you're checking or comparing. FQL values can be:

- `null`
- `true`
- `false`
- a numeral (integer), such as `1`, `500`, or `8675309`
- an alphanumeric string surrounded by single quote characters (`'`). If the string contains single quote (`'`) or double quote (`"`) characters, you must precede them with the backslash escape character (`\`). Strings can contain spaces ().

Values can also include wildcards (*).

Special characters

These special characters must be escaped if used in an FQL expression:

Special character	Definition
:	filter expression separator
+	"and" combination operator
,	"or" combination operator
(	left parenthesis
)	right parenthesis
!	"not" operator
>	"greater than" operator
>=	"greater than or equal to" operator
<	"less than" operator
<=	"less than or equal to" operator
[	left bracket
]	right bracket
'	single quote
"	double quote

Sorting with FQL

An FQL sort expression is a string in this format: `sort:<property>.<direction>`

- `:` is required and separates the sort command from the property and direction
- `<property>` represents the item you want to filter on, such as an attribute of a detection (`last_behavior_time`) or a host (`platform_name`)
- `.` is required and separates the property from the direction
- `direction` is either `asc` (ascending) or `desc` (descending)

For example, if you're using the CrowdStrike API to get a list of hosts, you could sort the hosts by the time they last contacted the CrowdStrike Cloud:

```
sort:last_seen.desc
```

Combining FQL expressions

You can join entire FQL expressions using these operators:

- `+` AND
- `,` OR

You can also use parentheses to override operator precedence: (`<filter expression>`). We recommend using spaces around expression or combination operators to improve readability. For example, all of these are valid and equivalent to a query where featured equals true AND the posts count is greater than 10 OR tags equals photo OR tags equals video:

- `featured:true+posts.count:>10, tags:photo,tags:video`
- `featured:true + posts.count:>10, tags:photo, tags:video`
- `featured: true + posts.count :> 10, tags:photo, tags:video`

Using timestamps

Timestamp filters use ISO 8601 format (YYYY-MM-DDTHH:mm:ss.sssZ). The timezone is always UTC as denoted by the suffix "Z".

Full timestamp value example: `'2006-01-02T15:04:05.2Z'`

Searching partial timestamp values

When entering a timestamp value, the full timestamp is not required. You can supply various full and partial date and time combinations.

- Full date: `YYYY-MM-DD`
- Partial date: `YYYY-MM`, `YYYY`
- Full date with partial time: `YYYY-MM-DDThh`, `YYYY-MM-DDThh:mm`, `YYYY-MM-DDThh:mm:ss`
- Partial date with full time: `YYYY-MMThh:mm:ss.sZ`, `YYYYThh:mm:ss.sZ`
- Partial date with partial time: `YYYY-MMThh`, `YYYY-MMThh:mm`, `YYYY-MMThh:mm:ss`, `YYYYThh`, `YYYYThh:mm`, `YYYYThh:mm:ss`

Examples

Date only:

- Get vulnerabilities with the hosts last seen after October 12, 2021:

```
filter=host_last_seen_timestamp:>'2021-10-12'
```

Date and hour:

- Get vulnerabilities last updated before October 12, 2021 at 3:00 UTC:

```
filter=updated_timestamp:<'2021-10-12T03'
```

Date plus hour and minute:

- Get vulnerabilities created after October 12, 2021 at 3:27 UTC:

```
filter=created_timestamp:>'2021-10-12T03:27'
```

Using comparison operators with timestamps

Always use a < or > comparison operator before a timestamp value. The equality operator is not supported; you cannot filter for exact timestamp values.

< (Occurred before the timestamp value)

- Get vulnerabilities last updated before 10/12/2021:

```
filter=updated_timestamp:<'2021-10-12'
```

> (Occurred after the timestamp value)

- Get vulnerabilities created after January 25, 2021 at 10:36:12 PM UTC:

```
filter=created_timestamp:>'2021-01-25T22:36:12Z'
```

You can use **now** for the current local timestamp value in your queries.

Example

- Get vulnerabilities created in the last 15 days:

```
filter=created_timestamp:>='now-15d'+created_timestamp:<'now'
```

Use cases and examples

Hosts API

These examples use the Host and Host Group Management APIs.

Tools and Reference

- Filter on devices that have been active in the last 7 days (assuming it is July 31):

```
$ curl -G -X GET -H "Content-Type: application/json" -H 'Authorization: bearer eyJhbGciOi4uXG1NNI' "https://api.crowdstrike.com/devices/queries/devices/v1" --data-urlencode "filter=last_seen:>'2021-07-24'"
```

- Filter on devices that were seen between July 23 and July 28:

```
$ curl -G -X GET -H "Content-Type: application/json" -H 'Authorization: bearer eyJhbGciOi4uXG1NNI' "https://api.crowdstrike.com/devices/queries/devices/v1" --data-urlencode "filter=last_seen:>'2021-07-23'+last_seen:<'2021-07-28'"
```

- Filter with wildcards on devices that start with the string "my-host-na":

```
$ curl -G -X GET -H "Content-Type: application/json" -H 'Authorization: bearer eyJhbGciOi4uXG1NNI' "https://api.crowdstrike.com/devices/queries/devices/v1" --data-urlencode "filter=hostname:*'my-host-na*'"
```

Note: The operator for wildcards, an asterisk, is required as well as one or more wildcard asterisks. The operator indicates that this is a wildcard search instead of the default "is equal to."

- Filter on devices that have been active in the last 7 days (assuming it is July 31) AND devices where the platform is "Windows":

```
$ curl -G -X GET -H "Content-Type: application/json" -H 'Authorization: bearer eyJhbGciOi4uXG1NNI' "https://api.crowdstrike.com/devices/queries/devices/v1" --data-urlencode "filter=last_seen:>'2021-07-24'"+platform_name:'Windows'"
```

- Filter on workstations with a containment status of "normal" AND whose platform name is "Windows" OR "Mac", AND which has been seen on or after July 4, 2021:

```
$ curl -G -X GET -H "Content-Type: application/json" -H 'Authorization: bearer eyJhbGciOi4uXG1NNI' "https://api.crowdstrike.com/devices/queries/devices/v1" --data-urlencode "filter=product_type_desc:'Workstation'+status:'normal'+platform_name:['Windows','Mac']+last_seen:>='2021-07-04'"
```

Kernel support info API

Examples using the kernel support API:

- Show support info for all Ubuntu 20 kernels that have a release starting with **5.8** and have a flavor of **gcp**:

```
vendor:'ubuntu'+distro:'ubuntu20'+flavor:'gcp'+release:*'5.8.*'
```

Note: The operator for wildcards, an asterisk, is required as well as the wildcard asterisks. The operator indicates that this is a wildcard search instead of the default "is equal to."

- Assume the current N-2 sensor version is **6.26.12304**. Find out if that sensor version supports a kernel release of **4.1.12-61.47.1.el6uek.x86_64**

```
release:'4.1.12-61.47.1.el6uek.x86_64'+  
(base_package_supported_sensor_versions:*'12304',ztl_supported_sensor_versions:*'12304')
```

- Show support info for kernels with a release of **5.4.0-1040-gcp** and an architecture of **x86_64**

```
release:"5.4.0-1040-gcp"+architecture:"x86_64"
```

For more information about the kernel support API, see [Retrieving kernel support info](#).